Learning Challenge: Data Analysis

The Scenario:

You are a data analyst intern working for a technology company. Your supervisor is preparing a report on how people use an AI-powered platform and wants to identify the most common types of user messages over time.

Before giving you access to the company's internal data, your supervisor has assigned you a practical exercise using a publicly available sample dataset.

The dataset contains information about different message topics and the proportion of messages belonging to each topic during a particular month.

Your supervisor wants a reliable Python function that can analyze this information and identify the most common message topic for any selected month.

You are encouraged to use an AI coding assistant to help develop the function. However, your supervisor expects you to provide clear instructions, identify potential problems, and explain how you would verify the accuracy of the AI-generated solution.

Your work will be assessed against a set of predetermined requirements and reference results.

The Challenge:

Your challenge is to use AI-assisted programming to solve a realistic data-analysis problem.

Rather than simply asking an AI assistant to analyze the data, you must formulate a clear and detailed programming request.

Your instructions should communicate what the function must accomplish, what data it will receive, what results it should produce, and how unexpected situations should be handled.

The objective is not merely to obtain working Python code. It is to demonstrate that you can communicate technical requirements effectively and critically evaluate an AI-generated solution.

Understanding the Dataset

You will work with a CSV dataset containing monthly message-topic statistics.

Dataset: share_of_messages_by_topic_month.csv

The dataset contains three columns:

Column	Description
month	The month associated with each observation
topic	The category assigned to a group of messages
share_of_messages	The proportion of messages belonging to that topic, expressed as a number between 0 and 1

Sample Data

The following is a small excerpt illustrating the data structure.

month	topic	share_of_messages
2024-07-01	Multimedia	0.020
2024-07-01	Practical Guidance	0.261
2024-07-01	Writing	0.341

Note: The complete dataset contains additional topics and months. This excerpt is not the complete data for July 2024.

A proportion of 0.341 represents 34.1% of messages.

Your solution should be reusable and work with different months, rather than being designed exclusively for July 2024.

Learning Objectives

By completing this challenge, you should be able to:

1. Translate a real-world data-analysis problem into clearly defined programming requirements.
2. Write precise instructions for an AI coding assistant, including inputs, outputs, constraints, and expected behavior.
3. Demonstrate an understanding of how pandas and NumPy can be used to filter and analyze numerical data.
4. Identify potential problems and edge cases that a programming solution must address.
5. Design appropriate tests and verification procedures to evaluate AI-generated code.
6. Critically assess whether a proposed solution satisfies the original requirements instead of assuming that AI-generated output is correct.

Your Task

Write a detailed prompt instructing an AI coding assistant to create a reusable Python function that satisfies the following requirements.

Requirement 1: Accept the necessary inputs

The function must accept a pandas DataFrame containing the dataset and a selected month, supplied in YYYY-MM format.

Requirement 2: Filter the dataset

The function must identify the rows corresponding to the requested calendar month. It should correctly handle dates stored in the dataset's month column.

Requirement 3: Identify the most common topic

The function must use NumPy, including np.argmax(), to identify the highest message share for the selected month.

Requirement 4: Convert proportions to percentages

The function must convert the highest share into a percentage and round it to one decimal place.

Requirement 5: Handle missing months

If the requested month does not exist in the dataset, the function must raise a ValueError with an informative message rather than producing an incorrect result.

Requirement 6: Handle equal shares

If multiple topics share the highest proportion, the function must return all tied topics, sorted alphabetically.

Requirement 7: Preserve the original data

The function must not modify the original DataFrame supplied by the user.

Requirement 8: Define the output

The function must return a Python dictionary containing:

- month: The selected month in YYYY-MM format.
- topics: A list of the topic or topics with the highest share.
- share_percentage: The highest message share expressed as a percentage, rounded to one decimal place.

Requirement 9: Provide verification

The AI-generated solution must include tests or verification examples covering normal input, a missing month, and multiple topics with equal shares.

Expected Results

Your supervisor has provided reference results that can be used to verify whether the solution behaves as expected.

Test Case 1: Normal Operation

Input: The supplied dataset, with July 2024 selected.

Expected result:


```
{
   "month": "2024-07",
   "topics": ["Writing"],
   "share_percentage": 34.1
}
```


Test Case 2: Missing Month

Input: A selected month that does not exist in the dataset.

Expected behavior: The function raises a ValueError explaining that no data is available for the selected month.

Test Case 3: Equal Message Shares

Input: A test DataFrame in which two topics have the same highest message share.

For example, suppose both Coding and Writing represent 40% of messages for a selected month.

Expected behavior: The function returns both topics in alphabetical order, with a share percentage of 40.0.

These examples establish the required behavior. Your task is to communicate these requirements clearly enough that an AI coding assistant can implement and verify them.

What You Must Submit

Submit a written response containing the following two components.

Part A: Your AI Programming Prompt

Write the prompt you would give to an AI coding assistant.

Your prompt must clearly communicate the programming problem, required Python libraries, expected inputs and outputs, important constraints, and the verification requirements.

The prompt should provide enough information to guide the AI without relying on unspecified assumptions.

Part B: Your Verification Plan

Briefly explain how you would determine whether the code produced by the AI is correct.

Your explanation should address:

- Which outputs you would compare against the reference results.
- How you would verify the function's behavior when a month is missing.
- How you would check the handling of equal message shares.
- Why it is important to verify AI-generated code instead of assuming that it works.

Important: Do not claim that tests were successfully executed unless you have actually run them and have evidence of the results.

Tips and Hints

Tip 1: Your prompts matter.

Be specific when communicating with an AI coding assistant. Clearly define inputs, outputs, constraints, and expectations. Vague instructions leave room for assumptions that may lead to incorrect or incomplete solutions.

Tip 2: Think about unexpected situations.

A function may work correctly for an ordinary dataset but fail when it encounters missing information or equal values. Consider these situations before asking the AI to generate code.

Tip 3: Understand the libraries you are using.

Pandas is useful for organizing and filtering tabular data, while NumPy provides numerical operations such as identifying the position of a maximum value.

Research the relevant functionality when necessary rather than relying entirely on the AI assistant's explanation.

Tip 4: Check the results.

AI-generated code is not guaranteed to be correct. Compare its output with known reference results and examine whether it satisfies every requirement.

Tip 5: Ask for evidence.

A strong solution should include meaningful tests, clear explanations, and verifiable results—not simply a statement that the code works.

Recommended Resources

You may consult the following official documentation:

- NumPy: numpy.argmax() — Identifying the index of a maximum value.
- pandas: to_datetime() — Converting date information into a usable format.
- pandas User Guide — Working with DataFrames and filtering data.

Completion Criteria

Your submission will be evaluated on whether it:

1. Communicates the programming problem and expected inputs clearly.
2. Specifies the appropriate pandas and NumPy operations.
3. Defines the required output and numerical formatting.
4. Addresses missing months, tied results, and preservation of the original data.
5. Provides meaningful verification requirements.
6. Demonstrates an understanding of why AI-generated solutions must be checked.

Successful completion requires more than mentioning the right programming terms. Your instructions must be sufficiently detailed and consistent to support a reliable, verifiable implementation.